Introduction to Python Programming (CS1005-2)

(Functions Overview, arguments and return values;formal vs actual arguments)

FUNCTIONS:

- ***Function is a sub program which consists of set of instructions used to perform a specific task. A large program is divided into basic building▪blocks called function.***

- **<u>Need For Function:</u>**

❖ When the program is too complex and large they are divided into parts. Each part is separately coded and combined into single program. Each subprogram is called as function.

❖ Debugging, Testing and maintenance becomes easy when the program is divided into subprograms.

❖ Functions are used to avoid rewriting same code again and again in a program.

❖ Function provides code re-usability

❖ The length of the program is reduced.

- **<u>Types of function:</u>**

Functions can be classified into two categories:

i) Built in function

ii) user defined function

i) Built in functions

❖ Built in functions are the functions that are already created and stored in python.

These built in functions are always available for usage and accessed by a programmer. It cannot be modified.

Built in function	Description
>>>max(3,4) 4	# returns largest element
>>>min(3,4) 3	# returns smallest element
>>>len("hello") 5	#returns length of an object
>>>range(2,8,1) [2, 3, 4, 5, 6, 7]	#returns range of given values
>>>round(7.8) 8.0	#returns rounded integer of the given number
>>>chr(5) \x05'	#returns a character (a string) from an integer
>>>float(5) 5.0	#returns float number from string or integer
>>>int(5.0) 5	# returns integer from string or float
>>>pow(3,5) 243	#returns power of given number
>>>type(5.6) <type 'float'>	#returns data type of object to which it belongs
>>>t=tuple([4,6.0,7]) (4, 6.0, 7)	# to create tuple of items from list
>>>print("good morning") Good morning	# displays the given object
>>>input("enter name: ") enter name : George	# reads and returns the given string

ii)**User Defined Functions:**

❖ User defined functions are the functions that programmers create for their requirement and use.

❖ These functions can then be **combined to form module** which can be used in other programs by importing them.

❖ Advantages of user defined functions:

· Programmers working on large project can divide the workload by making different functions.

· If repeated code occurs in a program, function can be used to include those codes and execute when needed by calling that function.

Function definition: (Sub program)

- def keyword is used to define a function.
- Give the function name after def keyword followed by parentheses in which arguments are given.
- End with colon (:)
- Inside the function add the program statements to be executed
- End with or without return statement

- **Syntax**:

```
def fun_name(Parameter1,Parameter2…Parameter n):
  statement1
  statement2…
  statement n
  return[expression]
```

- **Example:**

```
def my_add(a,b):
  c=a+b
  return c
```

Function Calling:

- Once we have defined a function, we can call it from another function, program or even the Python prompt.
- To call a function we **simply type the function name with appropriate arguments.**

- Example:

```
x=5
y=4
my_add(x,y)
```

Flow of Execution:

- The order in which statements are executed is called the **flow of execution**
- Execution always begins at the first statement of the program
- Statements are executed one at a time, in order, from top to bottom.
- Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.
- Function calls are like a bypass in the flow of execution. Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

Note: When you read a program, don't read from top to bottom. Instead, follow the flow of execution. This means that you will read the **def** statements as you are scanning from top to bottom, but you should skip the statements of the function definition until you reach a point where that function is called.

Parameters And Arguments:

Parameters(Formal Parameters):

- Parameters are the value(s) provided in the parenthesis when we write function header.
- These are the values required by function to work.
- If there is more than one value required, all of them will be listed in parameter list separated by **comma.**
- Example: def my_add**(a,b):**

Arguments(Actual Parameters) :

- Arguments are the value(s) provided in function call/invoke statement.
- List of arguments should be supplied in same way as parameters are listed.
- Bounding of parameters to arguments is done 1:1, and so there should be same number and type of arguments as mentioned in parameter list.
- Example: my_add(x,y)

Difference between Parameter and argument:

Aspect	Parameter	Argument
Definition	A variable listed in the function definition.	A value passed to the function when it is called.
Location	Appears in the function definition.	Appears in the function call.
Purpose	Acts as a placeholder for values that will be received.	Provides actual data for function execution.
Example	**def greet(name):** where name is a parameter.	**greet("Alice")** "Alice" is an argument.
Types	Positional, Default, Variable-Length	Positional, Keyword, Default, and Variable-Length arguments.

RETURN STATEMENT:

- The **return statement is used to exit a function** and go back to the place from where it was called.
- If the return statement has no arguments, then it will not return any values. But exits from function.

- **Syntax:**

  return[expression]

- **Example:**

```
def my_add(a,b):
  c=a+b
  return c
x=5
y=4
print(my_add(x,y))
```

- **Output:**

```
9
```

ARGUMENTS TYPES:

1. Required Arguments
2. Keyword(named) Arguments
3. Default Arguments
4. Variable length Arguments

1. Required Arguments:

- The number of arguments in the function call should match exactly with the function definition.

```
def my_details( name, age ):
print("Name: ", name)
print("Age ", age)
return
my_details("george",56)
```

- **Output:**

Name: george
Age 56

2. Keyword(named) Arguments:

- Python interpreter is able to use the keywords provided to match the values with parameters even though if they are arranged in out of order.

```
def my_details( name, age ):
print("Name: ", name)
print("Age ", age)
return
my_details(age=56,name="george")
```

- **Output:**

Name: george
Age 56

3. Default Arguments:

- Assumes a default value if a value is not provided in the function call for that argument.

```
def my_details( name, age=40 ):
print("Name: ", name)
print("Age ", age)
return
my_details(name="george")
```

- **Output:**

Name: george
Age 40

4. Variable length Arguments

- If we want to specify more arguments than specified while defining the function, variable length arguments are used.
- It is denoted by * symbol before parameter.

```
def my_details(*name ):
print(*name)
my_details("rajan","rahul","micheal",ärjun")
```

- **Output:**

rajan rahul micheal ärjun

Recursive Function

A Python function can be called from within its body. When we do so it is called a recursive function.

```
def fun( ) :
    # some statements
    fun( )    # recursive call
```

Recursive call keeps calling the function again and again, leading to an infinite loop.

A provision must be made to get outside this infinite recursive loop. This is done by making the recursive call either in if block or in else block as shown below:

```
def fun( ) :
    if condition :
        # some statements
    else
        fun( )    # recursive call
```

```
def fun( ) :
    if condition :
        fun( )
    else
        # some statements
```

- Variables created in a function die when control returns from a function.
- Recursive function may or may not have a return statement.
- Typically, during execution of a recursive function many recursive calls happen, so several sets of variables get created. This increases the space requirement of the function.
- Recursive functions are inherently slow since passing value(s) and control to a function and returning value(s) and control will slow down the execution of the function.
- Recursive calls terminate when the base case condition is satisfied.

Recursive Factorial Function

```
def refact(n) :
    if n == 0 :
        return 1
    else :
        p = n * refact(n - 1)
    return p

num = int(input('Enter any number: '))
fact = refact(num)
print('Factorial value = ', fact)
```

Lambda Function

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

- `lambda` *`arguments`* `:` *`expression`*

- ```
  x = lambda a : a + 10
  print(x(5))
  ```
  ```

- ```
  def myfunc(n):
    return lambda a : a * n
  ```

- ```
  def myfunc(n):
    return lambda a : a * n

  mydoubler = myfunc(2)

  print(mydoubler(11))
  ```

- **MODULES:**

➢ **A module is a file containing Python definitions, functions, statements and instructions.**

➢ Standard library of Python is extended as modules.

➢ **To use these modules in a program, programmer needs to import the module.**

➢ Once we import a module, we can reference or use to any of its functions or variables in our code.

- There is large number of standard modules also available in python.
- Standard modules can be imported the same way as we import our user-defined modules.
- Every module contains many function.
- *To access one of the function , you have to specify the name of the module and the name of the function separated by dot .* **This format is called dot notation.**

- **Syntax:**

```
import module_name
module_name.function_name(variable)
```

- **Importing Builtin Module:**

```
import math
x=math.sqrt(25)
print(x)
```

- **Importing User Defined Module:**

```
import cal
x=cal.add(5,4)
print(x)
```

There are **four ways to import a module** in our program, they are

Import: It is simplest and most common way to use modules in our code.
Example:
`import math`
`x=math.pi`
`print("The value of pi is", x)`
Output: The value of pi is 3.141592653589793
	from import : It is used to get a specific function in the code instead of complete file.
	Example:
	`from math import pi`
	`x=pi`
	`print("The value of pi is", x)`
	Output: The value of pi is 3.141592653589793

import with renaming:
We can import a module by renaming the module as our wish.
Example:
`import math as m`
`x=m.pi`
`print("The value of pi is", x)`
Output: The value of pi is 3.141592653589793
	import all:
	We can import all names(definitions) form a module using *
	Example:
	`from math import *`
	`x=pi`
	`print("The value of pi is", x)`
	Output: The value of pi is 3.141592653589793

- Built-in python modules are,

1.math – mathematical functions:

- some of the functions in math module is,
- math.ceil(x) - Return the ceiling of *x*, the smallest integer greater than or equal to *x*
- math.floor(x) - Return the floor of x, the largest integer less than or equal to x.
- math.factorial(x) -Return x factorial.
- math.gcd(x,y)- Return the greatest common divisor of the integers a and b
- math.sqrt(x)- Return the square root of x
- math.log(x)- return the natural logarithm of x math.log10(x) – returns the base-10 logarithms math.log2(x) - Return the base-2 logarithm of x. math.sin(x) – returns sin of x radians
- math.cos(x)- returns cosine of x radians
- math.tan(x)-returns tangent of x radians
- math.pi - The mathematical constant π = 3.141592 math.e – returns The mathematical constant e = 2.718281